

Ejercicio 7

Introducción

Este ejercicio busca implementar el ejemplo que trae **ESP-IDF** para usar la cámara del **ESP-CAM**, haciendo ligeras modificaciones para entender mejor el código y que imprima la foto en Hex para luego pasarlo por un código en Python y mostrar la imagen reconstruida.

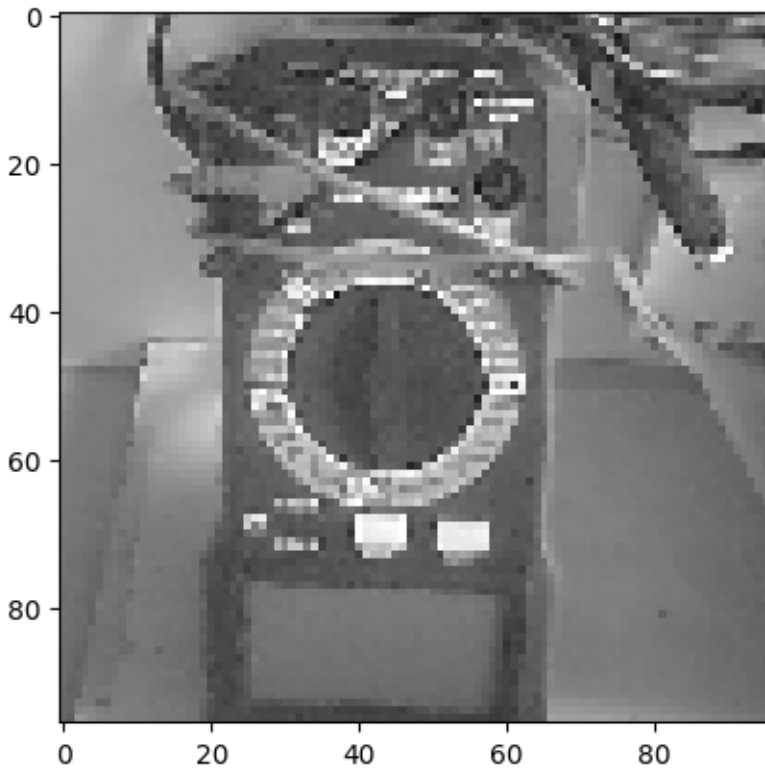
Funcionamiento

El código es, básicamente, la implementación base del ejemplo que entrega ESP-IDF para usar el ESP-CAM. Para esto se tuvo que configurar el MCU que en nuestro caso es la BOARD_ESP32CAM_AITHINKER. Luego se editó ligeramente el código para 2 cosas. En primer lugar el código está imprimiendo el clásico mensaje de los lugares con cámaras de vigilancia que dice "Sonrie, te estamos grabando", y después se hizo que el código imprimiera en consola los valores hexadecimales de cada pixel de la foto (se hizo que saltara la linea cada 24 pixeles). Por lo tanto, un output esperado sería el siguiente:

```
rairb@victus-huracan: ~/uan x + v
I (207) cpu_start: Pro cpu start user code
I (208) cpu_start: cpu freq: 160000000 Hz
I (209) app_init: Application Information:
I (213) app_init: Project name: Ejercicio_7
I (217) app_init: App version: aa435e5
I (221) app_init: Compile time: Apr 1 2026 13:53:38
I (225) app_init: ELF file SHA256: 013ca13ea
I (231) app_init: ESP-IDF: v6.1-dev-2938-g12f36a021f-dirty
I (237) efuse_init: Min chip rev: v0.0
I (241) efuse_init: Max chip rev: v3.99
I (245) efuse_init: Chip rev: v3.1
I (249) heap_init: Initializing, RAM available for dynamic allocation:
I (255) heap_init: At 3FFA6000 len 00001920 (6 KiB): DRAM
I (260) heap_init: At 3FFB2000 len 00020000 (127 KiB): DRAM
I (265) heap_init: At 3FFE9000 len 00003A00 (14 KiB): D/IRAM
I (270) heap_init: At 3FFC4300 len 00001C00 (11 KiB): D/IRAM
I (275) heap_init: At 40000000 len 00040000 (64 KiB): DRAM
W (282) spi_flash: Detected byte flash chip but using generic driver. For optimal functionality, enable 'SPI_FLASH_SUPPORT_BOYA_CHIP' in menuconfig
I (284) spi_flash: detected chip: generic
I (288) spi_flash: flash id: d10
W (301) spi_flash: Detected size(4096Ki) larger than the size in the binary image header(2048Ki). Using the size in the binary image header.
I (314) main_task: Started on CPU0
I (324) main_task: Calling app_main()
I (326) cam_hal: cam init ok
I (330) scdb-ng: pin_mux_24 pin_sel 27
I (334) scdb-ng: scdb_l2c_port=1
I (354) camera: Camera PID=0x26 VSR=0x02 MIDL=0x7f MIDH=0xa2
I (359) camera: Detected OV2640 camera
I (359) camera: Detected camera at address=0x30
I (424) cam_hal: PSRAM DMA mode disabled
I (428) esp32_ll_cam: node_size: 3072, nodes_per_line: 1, lines_per_node: 4, dma_half_buffer: 3072, dma_half_buffer: 12288, lines_half_buffer: 16, dma_buffer_size: 24576, image_size: 18432
I (430) cam_hal: buffer_size: 24576, half_buffer_size: 12288, node_buffer_size: 3072, node_cnt: 8, total_cnt: 1
I (440) cam_hal: Allocating 5216 byte frame buffer in OnBoard RAM
I (454) cam_hal: cam config ok
I (463) ov2640: Set PLL: clk_div: 0, clk_div: 2, pclk_auto: 1, pclk_div: 8
I (524) example_take_picture: Sonrie, te estamos grabando
0x1B, 0x18, 0x18, 0x17, 0x10, 0x11, 0x1B, 0x17, 0x13, 0x17, 0x17, 0x15, 0x19, 0x1D, 0x19, 0x14,
0x17, 0x11, 0x11, 0x17, 0x15, 0x12, 0x15, 0x1B, 0x12, 0x17, 0x13, 0x16, 0x16, 0x14, 0x11, 0x0F,
0x1A, 0x1A, 0x1B, 0x16, 0x14, 0x1C, 0x16, 0x0A, 0x15, 0x10, 0x0F, 0x10, 0x14, 0x14, 0x17, 0x16,
0x12, 0x12, 0x0F, 0x10, 0x16, 0x13, 0x0A, 0x0F, 0x0E, 0x18, 0x12, 0x19, 0x12, 0x11, 0x15, 0x13,
0x13, 0x13, 0x11, 0x0F, 0x11, 0x0F, 0x17, 0x12, 0x10, 0x0D, 0x14, 0x13, 0x12, 0x14, 0x0C, 0x0F,
0x11, 0x0E, 0x11, 0x0D, 0x0E, 0x13, 0x0A, 0x0F, 0x0F, 0x12, 0x10, 0x11, 0x13, 0x0E, 0x10, 0x10,
0x14, 0x15, 0x15, 0x13, 0x10, 0x0F, 0x12, 0x1C, 0x14, 0x15, 0x14, 0x19, 0x16, 0x12, 0x14, 0x10,
0x17, 0x11, 0x12, 0x12, 0x12, 0x14, 0x12, 0x13, 0x0D, 0x05, 0x16, 0x0D, 0x0E, 0x0E, 0x0F, 0x0E,
0x14, 0x0C, 0x11, 0x10, 0x17, 0x18, 0x05, 0x12, 0x09, 0x03, 0x11, 0x0B, 0x15, 0x14, 0x13, 0x0F,
0x12, 0x19, 0x07, 0x12, 0x0F, 0x0F, 0x07, 0x01, 0x0D, 0x11, 0x0E, 0x12, 0x10, 0x10, 0x11, 0x11,
0x0E, 0x0E, 0x09, 0x0A, 0x0F, 0x13, 0x08, 0x16, 0x0F, 0x0C, 0x13, 0x0A, 0x0F, 0x12, 0x0A, 0x10,
0x0D, 0x0F, 0x0D, 0x11, 0x0F, 0x09, 0x0D, 0x0B, 0x0D, 0x14, 0x0B, 0x10, 0x0A, 0x0D, 0x0B, 0x0A,
0x14, 0x22, 0x11, 0x1B, 0x19, 0x0D, 0x10, 0x2B, 0x0E, 0x11, 0x17, 0x11, 0x15, 0x14, 0x17, 0x11,
0x14, 0x10, 0x0F, 0x13, 0x14, 0x1B, 0x14, 0x15, 0x0E, 0x17, 0x12, 0x10, 0x10, 0x13, 0x09, 0x0B,
0x11, 0x15, 0x15, 0x0E, 0x15, 0x13, 0x0F, 0x17, 0x16, 0x12, 0x15, 0x13, 0x14, 0x10, 0x12, 0x13,
0x11, 0x13, 0x0E, 0x15, 0x12, 0x0E, 0x10, 0x11, 0x16, 0x0C, 0x0C, 0x10, 0x0E, 0x10, 0x15, 0x11,
0x17, 0x14, 0x12, 0x10, 0x0F, 0x0F, 0x0E, 0x10, 0x10, 0x13, 0x0F, 0x0D, 0x11, 0x0C, 0x0C,
0x11, 0x10, 0x0E, 0x0D, 0x12, 0x16, 0x0C, 0x0C, 0x13, 0x0B, 0x0C, 0x0C, 0x0A, 0x0A, 0x0B, 0x0D,
```

*(El output no está completo para no mostrar todos los pixeles)

Para poder ver la imagen, hay que copiar los valores entregados en el código y pegarlos en el notebook entregado por el profesor en el enunciado, que también está en este archivo .ipynb (o este Colab). Una vez cambiando los valores y ejecutando el notebook, podemos ver la foto. Aquí hay un ejemplo de un multímetro capturado con el ESP-CAM:



Aprendizajes

Este ejercicio muestra que el trabajar con imágenes imprimiendo los píxeles puede ser lento y que es importante aumentar el tiempo del watchdog para que no se cuelgue.